QCert: Verifying Quantum Circuit Completeness in Agda and Lean

ANONYMOUS AUTHOR(S)

A set of circuit equations is complete if every equation between circuits denoting the same linear map follows from it, and minimal if no equation can be derived from the others. Establishing these properties takes long case analyses spanning hundreds of pages, making the details hard to check by hand. To address this, we extend an existing formalization library to QCert in Agda and Lean. In QCert, humans set up a unified framework of circuits, matrix semantics, and group presentations, and state each paper's main theorem in it. LLM coding agents then write the proofs following the papers' arguments, and the proof assistant checks them. A human therefore need only review each paper's relations, gate semantics, and theorem statement. Using QCert, we first check sixteen published results, fifteen on completeness and one on minimality. They cover discrete gate sets such as Clifford, Clifford+T, and Toffoli–Hadamard, on qubits or qudits of every odd prime dimension. We verify fourteen of them directly and correct an error in each of the other two. We then prove a new result, generalizing the completeness of Toffoli–Hadamard circuits from three to any number of qubits. We expect QCert to help with other open completeness problems, such as those for quantum circuits in higher dimensions.

1 Introduction

Rewriting of quantum circuits is a ubiquitous tool in circuit optimization [36, 46, 47, 58], equivalence checking [59], hardware mapping [55], and reasoning about quantum programs [27]. This relies on identifying a set of rewrite rules for a family of quantum circuits that transform a circuit into an equivalent one. Such a theory should be *sound* so that every rewrite preserves the underlying linear map. It should also be *complete* so that any equivalence between circuits can be derived. Additionally, we want it to be *minimal*, with no redundant rules.

Completeness results have been established for a wide range of circuit families, including reversible linear circuits [38, 52], reversible circuits over multiple-control Toffoli gates [28, 29], Clifford [8, 41, 45, 54], CNOT-dihedral [1, 13], two-qubit Clifford+T [10], three-qubit Clifford+CS [11], Toffoli–Hadamard [2], and real Clifford+CH [21] circuits, as well as universal quantum circuits [23, 24]. Completeness has also been obtained for the group of two-level matrices underlying several of these circuit families [9, 20, 27, 31, 42]. Minimal theories have been given for six of these circuit fragments [14] and for all quantum circuits [22].

Completeness and minimality proofs typically require long case distinctions over gates, relations, and normal forms, whose supporting derivations sometimes span hundreds of pages [21, 27, 41]. In particular, relation reductions require intricate circuit manipulations and repeated rewrites, making them prone to human mistakes. For example, circular reasoning can lead us to remove a rule whose derivation still relies on that rule, hidden in a chain of other derivations. Errors can then propagate through dependent derivations, invalidating the entire reduction and making the error source hard to trace by hand.

The length of these proofs and their error-prone nature make it desirable to formally verify their correctness using a proof assistant like Lean or Agda. This would not have been feasible in the past due to the sheer number of lemmas and lengthy rewrites in these papers. Recent LLMs can write correct code for proof assistants, which brings this closer to reality. For example, Anthropic reports that LLMs wrote a 13-million-line Lean proof of Fermat's Last Theorem within 11 days [5]. However, care must still be taken. It is important to manually verify the *alignment* between the machine-verified statement and the intended natural-language result. For instance, [6] showed that a claimed verification of OpenAI's natural-language proof about the Navier–Stokes equations contained mistranslations, so the formal verification was verifying a weaker result by a different proof. A way to solve this is to use a hybrid approach, where humans write and carefully review the definitions and theorem statements [35], coding agents write the proofs, and Agda checks them.

Our contributions. In this work we present QCert, a framework in Agda [15, 49] and Lean [25] for representing and verifying quantum circuit completeness and minimality results, and use it to verify the correctness of sixteen completeness results. This is based on our formalization of the qudit Clifford completeness result of [8] (Section 4), where the framework [7] was built and the main verification steps were carried out by hand. Since all sixteen completeness results use the same framework of quantum circuits, gates, and matrices over a finitely generated ring, this makes it possible to efficiently check the alignment of the formalized results with the natural-language statements from the original papers. Of these sixteen results, we fully verify fourteen directly, and find fixes to verify the remaining two. For one result, we find a gap in its proof, which we fix, so the original completeness claim still holds. For another result, we find that the theorem is missing one relation, and we show that this relation is independent of the others, so completeness fails without it.

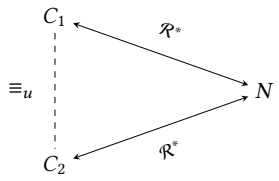
Orientation. In Section 2, we review the two main strategies for proving completeness of a rule set and how minimality is shown. In Section 3, we describe the QCert framework, which adds matrix semantics to the library of circuits and presentations of Bian and Feng [7], together with our formalization workflow, how QCert keeps completeness from being lost in translation, and a comparison of the Agda and Lean formalizations. In Section 4, we present the formalization of qudit Clifford completeness that QCert grew out of, and the mathematical choices it fixed. In Section 5, we use QCert to machine-check the completeness and minimality results of sixteen papers. In Section 6, we describe the issues these formalizations found in the source papers. In Section 7, we discuss the current limitations of QCert and our plans to extend it.

2 Background

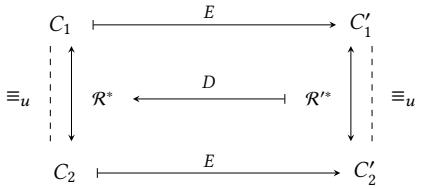
A circuit is a word over an alphabet $\mathcal{G}$ of gates. A rewrite rule is then a relation between words over this alphabet, and a rule set $\mathcal{R}$ generates a congruence on circuits. To a circuit C we associate an *interpretation* $[\![C]\!]$, which is its underlying matrix. The completeness results we formalize have matrix entries in the finitely generated rings $\mathbb{Z}\left[1/\sqrt{2}, i\right]$, $\mathbb{Z}\left[1/2, i\right]$,

$\mathbb{Z}\left[1/\sqrt{2}\right]$, and $\mathbb{Z}\left[1/2\right]$, which make them exactly representable in Agda. We leave the formalization of universal circuits taking values in $\mathbb{C}$ for future work.

There are two main strategies for proving completeness of a rule set $\mathcal{R}$ over a gate set $\mathcal{G}$, both of which we support. Consider any two circuits C_1 and C_2 over $\mathcal{G}$ that have the same linear map, $[\![C_1]\!] = [\![C_2]\!]$. We need to show that C_1 can be rewritten into C_2 using $\mathcal{R}$. Figure 1a outlines the first strategy. It specifies a unique normal form for each linear map [1, 8, 13, 28, 29, 38, 41, 45, 52, 54], often by giving an exact synthesis algorithm [9, 20, 27, 31, 42]. Completeness follows by proving that every circuit can be transformed into its normal form using the rewrite rules. By uniqueness, C_1 and C_2 have the same normal form N. Reversing the path from C_2 to N and concatenating it with the path from C_1 to N then gives the desired path of rewrites.



(a) The rewrites from C_1 and C_2 to their common normal form N using $\mathcal{R}$ give rewrites from C_1 to C_2 using $\mathcal{R}$.



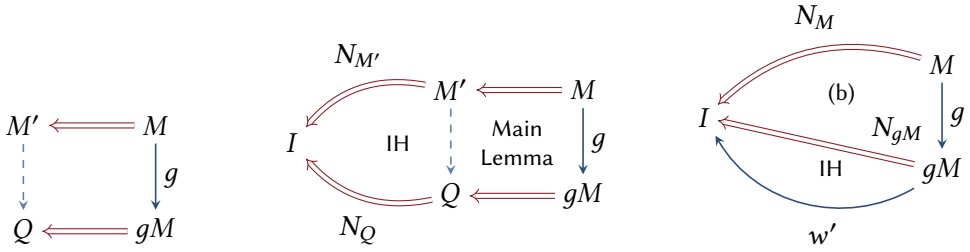
(b) Completeness of $\mathcal{R}'$ gives rewrites between $C_1' = E(C_1)$ and $C_2' = E(C_2)$. Translating this proof back gives rewrites between C_1 and C_2 using $\mathcal{R}$.

Fig. 1. Two strategies for proving completeness of $\mathcal{R}$ over a gate set $\mathcal{G}$. $\overset{\mathcal{R}^*}{\longleftrightarrow}$ means applying a finite sequence of rewrites using $\mathcal{R}$ in either direction, where $*$ denotes zero or more steps.

The exact synthesis results above present groups of matrices rather than circuits (the first block of Table 1). Their generators are *multi-level* matrices. For an $m \times m$ matrix M and distinct indices $a_1, \ldots, a_m$ in $\{0, \ldots, n-1\}$, the *m-level* matrix $M_{[a_1,\ldots,a_m]}$ is the $n \times n$ matrix that acts as M on the standard basis vectors $e_{a_1}, \ldots, e_{a_m}$ and as the identity on the others. For example, $(-1)_{[a]}$ negates e_a, and $\mathrm{X}_{[a,b]}$ exchanges e_a and e_b. Circuits on k qubits have matrices of dimension $n = 2^k$, but these groups are defined in every dimension n. We introduce multi-level matrices for two reasons. The gap we find in Section 6 is in a presentation by one- and two-level matrices [27], and our new result in **??** builds on a presentation by one-, two-, and four-level matrices [42].

For these groups, the normal forms come from exact synthesis, and the proof that every circuit can be transformed into its normal form has the same structure for each algorithm. Each matrix, or *state*, has a *level*, built from quantities such as the least denominator exponent of a column, and the algorithm reduces each state M to the identity in steps that lower the level. These steps form the normal word N_M. The *Main Lemma* says that for every state M and generator g, a diagram of relations closes the square formed by g and the algorithm's next steps from M and gM, passing only through states of lower level (Figure 2a). By well-founded induction on the level, the relations then rewrite $N_{gM}\, g$ into N_M (Figure 2b), and by induction on length, every word that reduces M to the identity rewrites into N_M (Figure 2c). Two words with the same matrix therefore rewrite into

each other. We formalize these inductions once and prove the Main Lemma for each presentation.



(a) The Main Lemma. (b) Induction on the level. (c) Induction on length.

Fig. 2. Completeness from the Main Lemma. Plain arrows are words, double arrows are steps or runs of the algorithm, and dashed arrows are words through states of lower level.

Figure 1b illustrates the second strategy, which uses an already complete rule set $\mathcal{R}'$ over a different gate set $\mathcal{G}'$ [2, 10–12, 14, 21, 23, 24, 26, 53]. Here, E and D translate circuits from $\mathcal{G}$ to $\mathcal{G}'$ and back, replacing each gate with a circuit over the other gate set. Both E and D preserve circuit composition and the underlying linear map. To transfer completeness from $\mathcal{R}'$ to $\mathcal{R}$, we first show that every rule of $\mathcal{R}'$, translated by D, can be derived using $\mathcal{R}$. We can then replace each step of the proof between C_1' and C_2' by its translated derivation, giving $D(C_1') \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_2')$. It then remains to prove that the rewrite $C_j \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_j')$ is derivable, for which it suffices to prove the round-trip rewrites $g \overset{\mathcal{R}^*}{\longleftrightarrow} D(E(g))$ for each $g \in \mathcal{G}$. Sometimes the two strategies are combined, as in the scaled dyadic extension of [42].

Starting from a complete rule set, we can remove a redundant rule by deriving it from the other rules, thereby preserving completeness. To show that the resulting rule set is minimal, we need to give, for each remaining rule R, an interpretation of circuits that every other rule preserves but R does not. Any derivation using the other rules preserves this interpretation, so R cannot be derived from them [14, 22].

3 The QCert Framework

Our Agda framework QCert states every completeness theorem as a group presentation. Unlike the letters of an ordinary word, gates act on wires. The gates in $\mathcal{G}$ are indexed by the number of wires they act on. Wires are numbered from 0 at the bottom, and a gate always acts on the bottom wires of a circuit. To place it anywhere else, we apply a shift, which takes a circuit on n wires and places it on wires $1, \ldots, n$ of a circuit with $n + 1$ wires. Each relation is stated on as many wires as the paper draws it on. To apply it inside a circuit with more wires, we need *structural rules*, which hold for every circuit, regardless of its gate set. These rules say that an equation can be shifted up by one wire, gates on disjoint wires commute, and a scalar can be placed on any wire. These define the equations of a PROP [43], and are often defined implicitly in completeness papers, but a formalization in Agda must define them explicitly.

We associate to circuits a matrix semantics $[\![-]\!]$ with entries in a finitely generated ring like $\mathbb{Z}\left[1/\sqrt{2}, i\right]$. Hence, $[\![-]\!]$ defines a group homomorphism to a unitary matrix group G. We build G from the group structures of Agda's standard library, which is checked only once per ring. As matrices, a gate on the bottom wires and a shift are Kronecker products with an identity, so the extension of $[\![-]\!]$ from gates to circuits is defined once for all gate sets. Since the structural rules are universal, their soundness is also proved only once. Then for each paper, it suffices to check the soundness of its own rules. Soundness and completeness say the relations present the image of $[\![-]\!]$, which is the subgroup of G generated by the gates. In Agda, mathematical structures such as groups are defined as records. The record $\mathcal{R}$ IsSubPresentationOf G contains the semantics $[\![-]\!]$, with proofs of soundness and completeness. The record $\mathcal{R}$ IsPresentationOf G adds surjectivity, so the relations present all of G.

Each formalization ends with its main theorem stated as an IsSubPresentationOf or IsPresentationOf record. For example, Statement (1) is the theorem for the real stabilizer circuits of [45], with the relations R1–R16 of its Figure 1 lifted to n wires. Statement (2) is the theorem for the two-level matrices of Hadamard-Π [27], with the relations of its Figure 6.

```
presentation : {n : ℕ} → (n VRel,_===_) IsSubPresentationOf U (2 ^ n)    (1)
presentation : {n : ℕ} → (_===_ {n}) IsPresentationOf (O n)              (2)
```

Agda checks every proof, so a reader only needs to check that each theorem statement matches its paper's result. Since every theorem is one of these two records, it suffices for a reader to check only four components: the relations transcribed from the paper, the target group, the matrix of each gate, and the line assigning $[\![-]\!]$ to the record. Each Agda development exports its main presentation theorem; the corresponding Lean statements are collected in `QCert/Table1.lean`.

3.1 Formalization Workflow

Our workflow has three parts. First, before formalizing a paper, we check its rewrite rules, its intermediate results, and each line of its derivations with exact matrices in short scripts. This caught most typos before any Agda code was written. Second, we state each main theorem in QCert before a coding agent starts, so the agent only supplies the proof. Finally, for long derivations, the agent writes a script that turns each line of the paper into one step of the Agda proof, so a failing step points to a specific line of the paper.

The main computational cost came from Agda's conversion checker. If two terms are syntactically identical (i.e., written in the same way), it accepts them immediately. Otherwise, it evaluates both, which can take exponential time for a long matrix product. For such products, we keep circuit interpretations abstract and decide matrix equality by computing a single Boolean using row operations, which is orders of magnitude faster than comparing matrices entry by entry. We record these techniques in the instructions followed by the agents, so later proofs avoid the same slowdowns.

3.2 Avoiding Completeness Lost in Translation

A machine-checked proof establishes only what its statement says. As the Navier–Stokes example in the introduction shows, a formalization can end up verifying a weaker result than the one intended [6]. For a completeness result, the meaning can be lost when the rules, the semantics, or the theorem is translated into a proof assistant. QCert addresses this in three ways.

Foundations. QCert builds on our formalization of qudit Clifford completeness, and keeps the two mathematical choices that Section 4 explains. The semantics is constructed independently of the circuit syntax. The target group is defined by its own mathematics, as a group of matrices or a semidirect product, and not as a quotient of circuits, so completeness relates two independently given objects and cannot hold by construction. Each formalization also states exactly what has and has not been formally verified. LLMs assist with proof construction and Agda checks the proofs, but we remain responsible for the definitions, the proof strategy, and the connection to the intended mathematics. Our contribution is thus not simply producing proofs that pass a checker, but understanding and justifying what those proofs establish. We use LLMs to extend our mathematical capabilities, not to outsource our mathematical judgment.

Explicit proof obligations. A proof by the second strategy of Section 2 starts from a set $\mathcal{R}$ of sound rewrite rules given by the source paper, and uses them to derive a larger rule set that is known to be complete (Figure 1b). This strategy suits QCert, because it turns completeness into explicit proof obligations, one for each rule of the larger set, which Agda either accepts or rejects. When an obligation cannot be discharged, say a rule K does not follow from $\mathcal{R}$, there are two possibilities. Either a derivation or a case is missing, and we supply it, or K must be added to $\mathcal{R}$. In the second case, the completeness claim concerns the revised rule set, and the formal statement says so. The relation r_{K} of Section 6 is an example. It does not follow from the rules of [2], so we added it and proved that it is independent of the others.

Shared structure. The unitary groups of different circuit families are mathematically related, often as subgroups of one another. The first five rows of Table 1 present four of these groups, and several circuit results are proved by translating into them. For example, two-qubit Clifford+T circuits are translated into $U_4(\mathbb{D}[\omega])$, three-qubit Clifford+CS circuits into $U_8(\mathbb{D}[i])$, and real Clifford+CH circuits into $U_{2^n}(\mathbb{D}[\sqrt{2}])$. A common framework therefore reflects this shared structure rather than merely collecting separate proofs. We verify general completeness arguments and algebraic constructions once, such as the inductions of Figure 2, Reidemeister–Schreier coset enumeration, and presentations of semidirect products, and reuse them by checking only the assumptions and semantic connections specific to each family. This reduces the need to translate each proof independently and makes the dependencies of each completeness claim explicit. Our contribution is not simply automating verification, but identifying the mathematical structure that makes verification systematic and reusable.

3.3 Comparison of the Agda and Lean Formalizations

The Lean [25] version of QCert addresses the same sixteen source results with a separate implementation. It makes the corpus available in the Lean ecosystem and builds on Mathlib [56]. The two implementations share the mathematical interface—generators, relations, semantics, and a presentation or sub-presentation theorem—but use different representations and sometimes different proof constructions. Their theorem statements therefore require separate alignment checks against the source papers.

Definitions and semantics. Agda uses a custom, explicitly bracketed word type and the standard library's setoid-based algebra: a presentation is a record containing a semantics and a group isomorphism, or monomorphism for a sub-presentation. Lean instead uses Mathlib's free monoid and generated monoid congruence, with presentation predicates in `Prop` and induced quotient-group isomorphisms. In both, the circuit infrastructure, structural rules, and general completeness arguments are shared between developments.

The semantic targets also differ. Most Agda developments work directly with exact algebraic representations of matrix entries. Lean states its matrix-group results over Mathlib's real or complex numbers, and proves bridges from computable exact rings to those targets. In particular, the Agda qudit Clifford development presents the constructed symplectic–Pauli–phase model described below Table 1, whereas Lean proves soundness and completeness against complex matrices at every symbolic odd prime dimension. Thus the common source corpus does not imply identical formal statements or intermediate lemmas.

Proofs and checking. Both implementations reuse normal-form, exact-synthesis, and translation arguments, and both check generated derivations by a proved replay procedure. Agda commonly discharges the resulting Boolean checks by reflexivity; Lean uses `decide +kernel`. In either case, the proof assistant checks the computation and the theorem connecting replay success to derivability. The scripts that search for derivations need not be trusted. The Agda Clifford+T and Clifford+CS developments reuse the authors' Agda proofs, as explained below. Lean constructs its own checked proof terms and certificate replays; its Clifford+T construction retains the authors' coset-action data, while its Clifford+CS construction supplies independently constructed coset certificates.

The trust checks reflect the assistants' different foundations. The Agda corpus uses safe modules without new postulates. The Lean corpus uses neither `native_decide` nor unfinished proofs in its main theorem chain: an audit of 105 declarations reports only `propext`, `Classical.choice`, and `Quot.sound`, with no `sorryAx` dependency.

Typechecking cost. Table 1 places the recorded Agda timings alongside a clean Lean build measured on 1 November 2026 at revision `62d6e3b3`. The Lean run used Lean 4.30.0, the pinned Mathlib revision, and WSL on the same Intel Core i5-14400 model, with eight logical CPUs, approximately 12 GiB of RAM, and 24 GiB of swap. We removed QCert's build artifacts while retaining the Mathlib and other dependency caches, then built folders sequentially in dependency order with `LEAN_NUM_THREADS=2`. Each folder's elapsed wall time includes Lake startup and Lean artifact generation; shared dependencies are counted only in the folder that defines them. These are build times rather than isolated kernel CPU times.

The run covered all 831 tracked library modules, excluding three generator templates, and took **12,573.26 seconds (3 h 29 min 33 s)** in timed build commands. Of these modules, 829 passed. Two unused legacy helpers in real Clifford+CH, `ItemBFun` and `ItemASteps`, failed respectively at `omega` proofs and the heartbeat limit; neither is imported by the main theorem chain or the audit, both of which passed. The real Clifford+CH entry in the table includes the full attempted folder build. Beyond the sixteen development folders, the shared `Circuit`, `Exact`, `Matrix`, and `NumberTheory` folders took 14.16, 13.90, 5.05, and 14.57 seconds, respectively; the five root modules, including the audit, added 54.83 seconds.

The Agda column retains the development-local measurements already reported in Table 1; it is not a new matched run. Differences in proof constructions, module boundaries, parallelism, and artifact generation prevent attributing the timings solely to the proof assistant. Memory pressure was especially significant in Lean's real Clifford+CH build, which incurred substantial swapping. Its five-wire certificate modules `Family267` and `Family268` alone took about 5,217 and 1,551 seconds. These measurements identify costs of the present implementations under the stated build conditions.

Since we have formalized the same results in both Agda and Lean, we can compare the two formalizations in three respects (Figure 3 and Table 2). These observations concern our current formalizations, not differences between Agda and Lean in general, and the two formalizations of a result often take different routes to it.

Proof development. We compare the code of the two formalizations (Figure 3(a), Table 2). Excluding blank lines and comments, the Agda formalization has about 376,000 lines and the Lean formalization about 340,000. Lean's is shorter in thirteen of the sixteen rows, by a factor of five or more for qudit Clifford, Toffoli–Hadamard and qutrit Clifford, and longer for Hadamard-Π, three-qubit Clifford+CS and real Clifford+CH. Both counts include the derivations and certificates that scripts generate. The Agda proofs are explicit terms, built by pattern matching and equational reasoning, with reflection-based solvers for associativity and ring identities, and its library develops the group theory the proofs need (about 13,100 lines, against 1,200 in Lean). The Lean proofs rely on tactics, with about 23,000 calls to automation such as `simp`, `omega`, `decide` and `ring`, and on the algebra of Mathlib. The logics differ too. Every Agda module is checked with `--safe` and uses no postulates, so the formalization is constructive. Every main theorem of the Lean formalization depends on `Classical.choice`, as the audit above shows.

Verification time. Figure 3(b) plots the check times of Table 1. In total Agda took 2,332 seconds and Lean 12,471, or 1,927 and 3,602 without real Clifford+CH, whose Lean build swapped heavily. Lean was faster for three rows, Clément's presentation, qudit Clifford and qutrit Clifford, whose Lean proofs take different routes (see *Definitions and semantics* above). Both assistants decide equalities of matrices and certificates by evaluation in the checker: Agda reduces a Boolean to `true`, and Lean uses `decide +kernel`. The two columns were measured under different conditions, and we have yet to separate the time Lean spends running proof automation from the time it spends checking the resulting proofs.

Execution time. The Agda normal forms are executable functions. The Lean formalizations obtain normal forms either as existence theorems, as for the Clifford and real

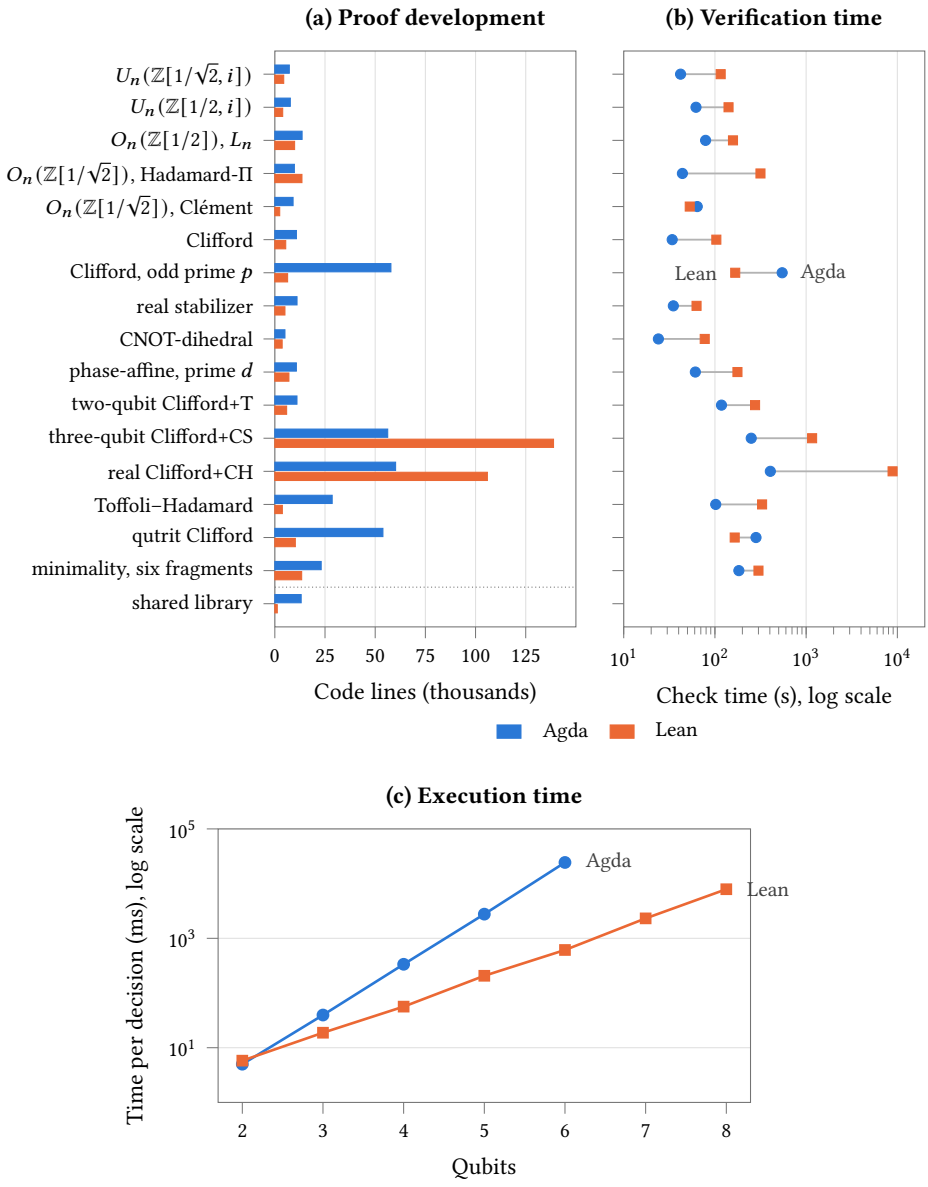


Fig. 3. The Agda and Lean formalizations of each row of Table 1. (a) Code lines, excluding blank lines and comments. The last row is the shared library of each framework. (b) The check times of Table 1, Agda (circles) and Lean (squares), measured under the different conditions described above. (c) The time each compiled formalization takes to decide that two Clifford circuits of 100 gates have the same matrix, by the procedure its soundness proof uses; medians over three runs of five random circuits.

stabilizer circuits of [45, 54], whose Lean proofs show that every circuit equals a normal form without defining a function that computes it, or as `noncomputable` definitions, as for the exact synthesis of the two-level groups; 160 definitions in the sixteen Lean developments are declared `noncomputable`. There is therefore no Lean normalizer to time. What both formalizations can execute is the procedure that decides whether two circuits have the same matrix, which their soundness proofs run. We compiled this procedure for the Clifford circuits of [54] to native code, through Agda's GHC backend and Lean's C backend, and timed both on the same random circuits of 100 gates over H, S and CZ, deciding that each circuit w has the matrix of $w\,S^4$ (Figure 3(c)). On two qubits Agda is slightly faster, 5.0 against 5.8 milliseconds per decision. From three qubits on Lean is faster, by a factor of 2.1 on three qubits and 40 on six, where Agda takes 24 seconds and Lean 0.61. The gap comes mostly from the procedures rather than the compilers: Agda computes each circuit's matrix as a product of stored $2^n \times 2^n$ matrices, so its work grows eightfold with each qubit, whereas Lean simulates the circuit on each basis vector, so its work grows fourfold.

4 The Foundation: Qudit Clifford Completeness

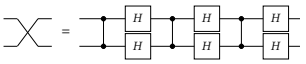
QCert grew out of our formalization of the completeness theorem of Bian et al. [8], henceforth *the qudit paper*. That theorem gives sixteen rewrite rules that are sound and complete for multi-qudit Clifford circuits over H, S, and CZ in every odd prime dimension p. Unlike the other fifteen formalizations, this one was built largely by hand, and it fixed the mathematical choices that the rest of QCert follows. It is checked in safe, axiom-free Agda against a semantics constructed independently of the syntax. The proof follows the paper's for the symplectic factor, where the paper does its real work, and diverges once, at the step that promotes symplectic completeness to projective completeness *up to Paulis* (Section 4.1). There we instead *factor the group, present the factors, and compose the presentations*. The symplectic group gets a coset-table normal form, the Pauli group a compositional presentation, and the scalars a cyclic one. Generic verified theorems for semidirect products and central extensions glue them, isomorphisms of presentations reduce the result to the paper's rules, and a factor $\langle -1 \rangle$ reaches its Figure 1 over $-\omega$.

4.1 Qudit Clifford Circuits and the Main Theorems

Fix an odd prime p and $\omega = e^{2\pi i/p}$. On the basis $|0\rangle, \ldots, |p-1\rangle$ indexed by $\mathbb{Z}_p$, $X|j\rangle = |j+1\rangle$ and $Z|j\rangle = \omega^j|j\rangle$ generate, with the phases ω^t, the Pauli group $\mathcal{P}_n$, and $H\colon |j\rangle \mapsto \frac{\delta_p}{\sqrt{p}} \sum_k \omega^{jk}|k\rangle$, $S\colon |j\rangle \mapsto \omega^{j(j-1)/2}|j\rangle$, $CZ\colon |j\rangle|l\rangle \mapsto \omega^{jl}|j\rangle|l\rangle$ generate, with the phases ω^t, the Clifford group C_n ($\delta_p = i^{(p-1)/2}$ makes the determinant of H equal to 1; the determinants of S and CZ are 1 too) [8]. Modulo scalars, $\overline{C}_n = C_n/\langle\omega\rangle$ and $\overline{\mathcal{P}}_n \cong \mathbb{Z}_p^{2n}$, $C_n/\mathcal{P}_n \cong \overline{C}_n/\overline{\mathcal{P}}_n \cong \mathrm{Sp}(2n, \mathbb{Z}_p)$, and $\overline{C}_n \cong \overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. These "glue facts" are the two not formalized: our semantic groups are *defined* as the right-hand sides. In [8] the global phases also include -1, so that the swap and the multiplier gates belong to C_n; we work mainly in the projective quotient, where -1 makes no difference except that the relations come out a little simpler.

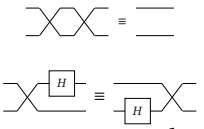
Using the framework of Section 3, we encode circuits, circuit relations, and reasoning with the relations. We draw circuits in matrix-multiplication order, the reverse of the usual circuit order: $w \bullet v$ applies v first but draws it on the right. For example, the swap:

```
Ex : Word (Gen (2+ n))
Ex = CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑
```



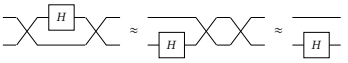
A rule set is an inductive family indexed by the width, one constructor per rule, each stated once at the smallest width at which it fits; for example, two of the fifteen constructors of our primary rule set _SRel,_===_, the order of the swap and the rule moving H across it:

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order-Ex     : (2+ n) SRel, Ex ^ 2 === ε
  semi-Ex-H↑ : (2+ n) SRel, Ex • H ↑ === H ↓ • Ex
```



And reasoning with the rules is equational reasoning in the congruence $\approx$ they generate, with combinators for symmetry, associativity, and rewriting the left or the right factor of a product; for example, conjugating by the swap moves H down a wire:

```
conj-Ex-H↑ : Ex • H ↑ • Ex ≈ H ↓
conj-Ex-H↑ = begin
  Ex • H ↑ • Ex    ≈⟨ sym assoc ⟩
  (Ex • H ↑) • Ex ≈⟨ cleft axiom semi-Ex-H↑ ⟩
  (H ↓ • Ex) • Ex ≈⟨ assoc ⟩
  H ↓ • Ex • Ex    ≈⟨ cright axiom order-Ex ⟩
  H ↓ • ε          ≈⟨ right-unit ⟩
  H ↓ ∎
```



Further, we can express and prove our main theorems:

Theorem 4.1 (clifford-presentation). *For every width n, the fifteen rules—the relations of [8], read modulo scalars—present the projective Clifford group:*

```
clifford-presentation : ∀ n → (PapV1.Clifford-Relations._QRel,_===_ n) IsPresentationOf (Pauli⋊Sp n)
```

QRel,===_ closes _SRel,_===_ under the three structural rules (Section 3); Pauli⋊Sp n is the semidirect product of $\mathbb{Z}_p^{2n}$ by $\mathrm{Sp}(2n, \mathbb{Z}_p)$ (Section 4.3); _IsPresentationOf_ (Section 3) is a group isomorphism $\mathrm{Cir}\ n/{\approx} \to \overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$: *soundness* (well-definedness of the map), each rule holds; *completeness*, [[_]] is injective; and *surjectivity*, each projective Clifford operator is represented by a circuit.

We refine the qudit paper's normal form for symplectic operators (Clifford operators modulo Paulis) into a doubly inductive definition (Section 4.2) and prove that it enjoys the unique-normal-form property:

Theorem 4.2 (unique-nf). *Normal forms with equal symplectic denotations are equal:*

```
unique-nf : ∀ n {u v : NF n} → [[ [ u ] ]] ≈ˢ [[ [ v ] ]] → u ≡ v
```

The route. We mostly follow the paper: its normal boxes, its symplectic normal form and its parametrized *box relations* — pushing a gate through a normal box yields an updated box and residual *dirty* gates — reappear below, with the reduction to eighteen relations of its

Theorem 4.4, a step the paper already delegated to Agda. The one major divergence is the step that promotes symplectic to projective completeness (its Proposition 4.9): the paper corrects each rule $C_l = C_r$ to $C_l = C_r\,;P$ for the unique Pauli P making it projectively sound and replays the symplectic rewrites, pushing each correction to the end, where it becomes "a potentially different Pauli". We did not see how to verify it at reasonable cost. We instead work in a syntax where Paulis are genuinely trivial — symplectic circuits have only H, S, CZ and denote $\mathrm{Sp}(2n, \mathbb{Z}_p)$ itself — and reintroduce Paulis and scalars by construction (Figure 4): the Pauli bookkeeping of Proposition 4.9 is done once, by a semidirect-product theorem proved for arbitrary groups before this project began.

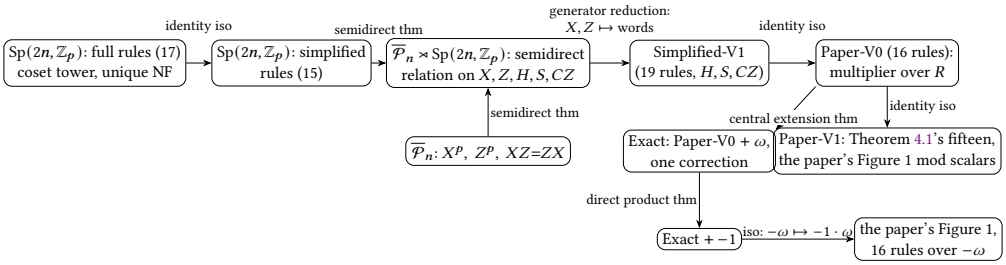
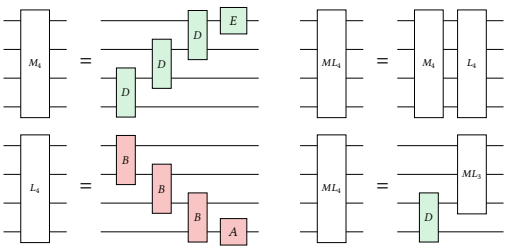


Fig. 4. The chain of presentations for qudit Clifford circuits: every arrow is a verified presentation theorem or isomorphism of presentations; the semantic group is $\mathrm{Sp}(2n, \mathbb{Z}_p)$ on the left, $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$ from the semidirect level on, a central extension of it by $\langle\omega\rangle$ at "Exact", and that times $\langle -1\rangle$ below it.

4.2 The Symplectic Factor

$\mathrm{Sp}(2n, \mathbb{Z}_p)$ is modeled as the group of $\mathbb{Z}_p$-linear, form-preserving bijections of the phase space Pauli n = Vec $(\mathbb{Z}_p \times \mathbb{Z}_p)$ n (the exponents of $Z^a X^b$ up to phase, per wire). Against this semantics, completeness rests on the normal form, which we refine into a doubly inductive datatype:

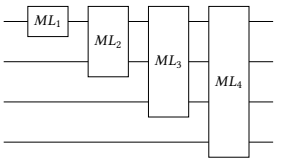
```
M L ML : ℕ → Set
M 0 = ⊤
M (1+ n) = Vec D n × E
L 0 = ⊤
L (1+ n) = Vec B n × A
ML 0 = ⊤
ML 1 = M 1 × L 1
ML m@(2+ n) = M m × L m
            ⊎ D × ML (1+ n)
```



An M-row is a column of D boxes ending in an E box (the paper's X-normal layer); an L-row is a column of B boxes ending in an A box (its Z-normal layer, with the A box at the bottom). An ML box at width $m \geq 2$ is *either* a full pair of rows *or* a bottom D box followed by an ML box one wire up: a spine of D boxes capped by a pair of rows. This is the first induction, on the width of a single coset representative. The second induction is the staircase:

```
NF : ℕ → Set
NF 0 = ⊤
NF (1+ n) = NF n × ML (1+ n)

[_] : ∀ {n} → NF n → Word (Gen n)
[_] {0}     tt       = ε
[_] {1+ n} (nf , ml) = [ nf ] ↑ • [ ml ]ml
```



The table. The coset table is the paper's "push a gate through a normal box" as a structurally recursive function ract (5 600 lines), with its soundness law ract-sound:

```
ract       : ML (1+ n) → Gen (1+ n) → Circuit n × ML (1+ n)
ract-sound : ∀ (c : ML (1+ n)) g → let (b' , c') = ract c g in [ c ]c • [ g ]w ≈ b' ↑ • [ c' ]c
```

Each clause of ract-sound is derived from the fifteen simplified symplectic relations; for instance, pushing S through an E box is one S-power step, $[\,b\,]^{\mathrm{e}} \bullet S \;=\; S^{-b} \bullet S \;\approx\; S^{-(b-1)} \;=\; [\,b-1\,]^{\mathrm{e}}$. A circuit is rewritten to its normal form by repeating this step, gate by gate; every rewrite step is thereby traced to a relation.

Uniqueness. The symplectic group acts on Vec $\mathrm{Pauli}_1\ n$, a $2n$-dimensional symplectic $\mathbb{Z}_p$-vector space in which Pauli_1 is a two-dimensional symplectic subspace, represented by a wire in a circuit. The elements fixing the bottom wire's subspace pointwise form a copy of $\mathrm{Sp}(2(n-1),\mathbb{Z}_p)$, acting above the bottom wire, and the ML boxes represent its cosets: every $f \in \mathrm{Sp}(2n,\mathbb{Z}_p)$ factors as $f = f' \circ c_n$ with $c_n \in ML_n$ and f' above the bottom wire, and c_n is unique because the bottom-wire output determines the box. Recursing on f' writes f uniquely as $c_1 \circ c_2 \circ \cdots \circ c_n$ with $c_k \in ML_k$, which is Theorem 4.2.

Surjectivity. Every symplectic transformation is the action of some circuit, proved constructively by "clear the bottom wire, recurse on the tail" — the paper's Lemma 3.7, its coset box computed rather than asserted:

```
Theorem-LM : ∀ {n} (p q : Pauli n) → sform p q ≡ 1 → ∃ λ (lm : ML n) → act [ lm ]ml p ≡ pZ0 × act [ lm ]ml q ≡ pX0
```

Completeness. Circuits with the same denotation rewrite through the table to their normal forms, which denote the same map by soundness and so are equal by Theorem 4.2: the circuits are equivalent. That argument is the framework's by-normalization (Figure 1a); with surjectivity, the simplified relations present the symplectic group:

```
simplified-presentation : ∀ n → (n SRel._===_) IsPresentationOf (Sp-group n)
```

4.3 Composing: Paulis, the Semidirect Product, and Scalars

$\overline{\mathcal{P}}_n \cong (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ is presented over X, Z on each wire by $X^p = Z^p = \varepsilon$ and $ZX = XZ$; its semantic group is the phase space that $\mathrm{Sp}(2n,\mathbb{Z}_p)$ acts on, and its normal form is $X^a Z^b$ per wire. Gluing this to the symplectic presentation of Section 4.2 is an instance of the framework's theorem for semidirect products: the presentation has both factors' relations plus the conjugation relations $h\,n\,h^{-1} = \mathit{conj}\,h\,n$, the action conj spelling out the paper's Lemmas 2.24–2.25. The theorem's two conditions — the action respects the relations of each factor — are here proved by semantics: the conjugates of congruent words have equal denotations by symplectic soundness, hence are congruent by Pauli

completeness (respects-Δ; the paper's "Pauli complete" rules, Definition 4.7). Every premise being a theorem, the semidirect presentation of `Pauli⋊Sp` is unconditional.

Plumbing. The semidirect presentation has generators X, Z, H, S, CZ and 3 + 15 + 14 relations (Pauli, symplectic, conjugation); the paper's rule sets have H, S, CZ alone. Three isomorphisms of presentations close the gap (Figure 4), through `Simplified-V1` (nineteen rules), `Paper-V0` (sixteen) and `Paper-V1` (Theorem 4.1's fifteen): generator reduction (X and Z become words) and relation reduction (the Pauli and conjugation rules become theorems). *Semidirect to Simplified-V1* is an isomorphism on *different* alphabets, f sending S to $R = S \bullet Z^{1/2}$ and h sending S to $Z^{-1/2} \bullet S$; in particular, f induces the embedding of the symplectic group into the projective Clifford group. `Simplified-V1`'s nineteen rules are the fifteen simplified symplectic ones plus $(SH)^3 = \varepsilon$, $H^2SH^2S = SH^2SH^2$ and the two rules pushing X through CZ; f-well-defined derives from them the images under f of the three Pauli and the fourteen conjugation rules (6 000 lines). *Simplified-V1 to Paper-V0* is the identity on circuits: ten axioms are shared, and V0's `Lemmas` (4 750 lines) derives c10–c15 from its own axioms. *Paper-V0 to Paper-V1* reaches the paper's own rules. The two rule sets differ in how the multiplier is spelled as a word: `Paper-V0` spells $M_x = R^xHR^{x^{-1}}HR^xH$ over R; the paper's (T1) spells $M_x = \mathsf{SHS'}\ x^{-1}\ x$ over S, with $\mathsf{SHS'}\ a\ b = Z^{(b-1)/2}X^{(1-a)/2}S^bHS^aHS^bH$ and semi-MR as $M_gS^{g^2}Z^{(g^2-g)/2} = SM_g$. `Paper-V1` restates the four multiplier rules over SHS' and drops one: M_1 is now the word $(SH)^3$, so M-power at $k = 0$ already gives $(SH)^3 \approx \varepsilon$, whereas over R it is $(RH)^3$, a different word, and the axiom had to stay. The bridge $R^aHR^bHR^aH \approx \mathsf{SHS'}\ a\ b$ (`Shared.PauliBase`) carries the multiplier rules across, giving Theorem 4.1.

Scalars. extension-presentation extends the projective presentation — `Paper-V0`'s, which has order-SH as an axiom — by the scalar: add ω and $\omega^p = \varepsilon$, let ω commute with every gate, and *correct* each rule by its scalar. The correction is ε except on order-SH, the one rule failing on the nose: $(SH)^3 = \omega^{-1/8}$, exponent $(p^2-1)/8 \equiv -1/8$ in $\mathbb{Z}_p$. The multiplier words now denote $|j\rangle \mapsto |xj\rangle$ only up to a scalar. The semantic extension is the Weyl cocycle on the natural semidirect product, $c((P,S),(Q,S')) = \frac{1}{2}\,\mathrm{sform}(P,\ S \cdot Q)$, with $S \cdot Q$ the symplectic part acting on the Pauli vector and sform the symplectic form of the phase space, the exponent in $PQ = \omega^{\mathrm{sform}(P,Q)}\,QP$; its cocycle identity is, by bilinearity, symplectic invariance of sform, and the presented group's cocycle is the pullback of c along Theorem 4.1's isomorphism, so the two agree by construction. The unitary identification uses Weyl multiplication [32, Eq. (4), Theorem 3] and a *linear* Weil representation with exact covariance [33, Theorem 1.3.2]: the operators $U(e,v,S) = \omega^e W(v)\rho(S)$ obey

$$U(e,v,S)U(f,w,T) = U\big(e + f + \tfrac{1}{2}\mathrm{sform}(v,Sw),\, v + Sw,\, ST\big).$$

This is the Heisenberg–Weil representation [34, §1.3]; conjugation on Weyl operators determines S and v, so it is faithful and its scalars are exactly $\langle \omega I\rangle$ (at width zero, $e \mapsto \omega^e$). Adjoining $-I$ gives its direct product with $\mathbb{Z}_2$ and scalars $\langle -\omega I\rangle$. These finite phase choices differ from the full unitary normalizer, which contains $U(1)$. Unconditionally:

```
presentation-exact : ∀ n → (n Exact._===_) IsPresentationOf (PE.Presented-group n (exact-generator-data n))
```

The extension is provably non-split.

Figure 1 as printed is over $-\omega$, its multipliers and swap carrying Legendre symbols and λ_p^2, so it needs the -1 that the exact group (scalars of odd order) lacks. Adjoining -1 as a central involution presents the exact group times $\mathbb{Z}_2$, Clifford±, and Figure 1 is isomorphic to that rule set by $-\omega \mapsto -1 \cdot \omega$, each exact rule's scalar pinned by reading its lift back: the paper's Theorem 4.10, against our group,

```
figure1-presentation : ∀ n → (n F,_===_) IsPresentationOf (Clifford± n)
```

4.4 Discussion and Related Work

Everything above is checked by Agda 2.8 with standard library 2.4 under `--safe`: no postulates, no holes. The development is 113 526 lines in 349 files: framework 15 000, qudit-specific 60 000 (symplectic 35 000, composition 23 000).

Not formalized are the two well-known *glue facts*: that the projective Clifford group is $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$, and that the Clifford group is a central extension of it by $\langle\omega\rangle$ (times $\langle -1\rangle$ over $-\omega$) [32, 33]; the representation argument above is external mathematics, and matching the paper's gate phases remains a convention check [8, 48]. No matrix appears in the development.

We have used versions of Claude to help with this formalization: composing box relations to prove ract-sound, defining the symplectic group, and establishing the equivalence between the code's relations and the qudit paper's Figure 1 modulo scalars; presentation-exact and the Figure 1 layer with -1 were done by Claude.

Related work. The completeness proofs for these gate groups, from Selinger's n-qubit Clifford presentation to the qudit rules, are pen-and-paper (Section 1); mechanized completeness is rare: Π as a free symmetric rig groupoid [19] and its square-root extension [17] are coherence arguments. Verified compilers and logics [18, 36, 39, 40, 50, 51, 60] prove soundness of rewrites, never completeness, and mathlib [56, 57], the AFP [16, 37] and MathComp [30] have no verified coset enumeration against a given concrete group.

Next: a verified matrix semantics over $U_{p^n}(\mathbb{Z}[1/p, \omega])$, which would close the chain and discharge the glue facts, and submission to the Agda standard library. The qubit case [54] and the qutrit rules of [41], which the odd-prime development does not cover, are now among the formalizations of Section 5.

5 Formalized Completeness and Minimality Proofs

The sixteen formalizations in Table 1 contain about 420,000 lines of Agda code and use no postulates, and each theorem is as general as the result in its source paper. Moreover, all matrices are computed exactly. Their entries lie in algebraic number rings, whose elements are given by finitely many integers, so Agda checks automatically whether two matrices are equal. Over the real or complex numbers, no such automatic check exists, which limits the equality checking of QCert to discrete gate sets, as we discuss in Section 7.

In the Agda development, most theorems in Table 1 are proved from scratch in QCert against a matrix semantics, with three exceptions. The two rows of [10, 11] build on the Agda proofs that accompany those papers. These proofs relate the circuit relations to two-level relations by the Reidemeister–Schreier method [44]. We connect them to our formalizations of the two-level matrix presentations in [9, 31], the first two rows of the

Source	Object	Result	Agda Mod	Agda time (s)	Lean time (s)
Groups of m-level matrices					
Greylyn [2014]	$U_n(\mathbb{Z}[1/\sqrt{2}, i])$	presentation, $n \leq 4$	31	42	115.62
Bian and Selinger [2021]	$U_n(\mathbb{Z}[1/2, i])$	presentation	37	62	140.99
Li et al. [2021]	$O_n(\mathbb{Z}[1/2])$, L_n	presentations	68	79	157.76
Fang et al. [2026]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	44	44	315.70
Clément [2019]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	40	64	52.94
Circuits					
Selinger [2015]	Clifford	sub-presentation	31	34	103.61
Bian, Li et al. [2026]	Clifford, odd prime p	presentation of a model	215	546	166.92
Makary et al. [2021]	real stabilizer	sub-presentation	30	35	62.87
Amy et al. [2018]	CNOT-dihedral	sub-presentation	30	24	77.29
Blake [2026b]	phase-affine, prime d	sub-presentations	56	61	176.59
Bian and Selinger [2023a]	two-qubit Clifford+T	sub-presentation in U_4	62	118	275.40
Bian and Selinger [2023b]	three-qubit Clifford+CS	sub-presentation in U_8	123	250	1162.66
Clément [2026]	real Clifford+CH	sub-presentation	346	405	8868.59*
Amy et al. [2024]	three-qubit Toffoli–Hadamard	presentation of L_8	201	102	328.84
Li et al. [2025]	qutrit Clifford	presentation	89	282	164.76
Minimality					
Blake [2026c]	six fragments	minimal, complete	73	183	300.20
Total			1,476	2,332	12,470.74

Table 1. Completeness and minimality results formalized in QCert. A presentation gives generators and relations for a whole group, and a sub-presentation for the subgroup generated by the gate set named in the Object column of the Circuits block. L_n is the group of orthogonal matrices $N/\sqrt{2}^j$, where N is an integer matrix and $j \geq 0$. Mod counts Agda modules. The Agda times are the previously recorded, rounded typechecking times from scratch with Agda 2.8 on an Intel Core i5-14400; the Lean times are elapsed folder-build times under the conditions of Section 3.3. Both columns exclude the shared library and other developments on which a row depends. The Result column describes the Agda statements; Section 3.3 explains the semantic differences. *Includes two unused legacy helpers that failed; the main theorem chain passed.

table, to obtain sub-presentations of $U_4\left(\mathbb{Z}\left[1/\sqrt{2}, i\right]\right)$ and $U_8\left(\mathbb{Z}\left[1/2, i\right]\right)$. The qudit Clifford theorem of [8] is the only one whose semantics is not a matrix group. Its group presentation lifts a presentation of the symplectic group $\mathrm{Sp}(2n, \mathbb{Z}_p)$ with Pauli corrections and global phases, and its semantics is built in the same way (Section 4).

	Agda	Lean
Proof development		
Logic	constructive: every module `--safe`, no postulates	classical: every main theorem uses `Classical.choice`
Library	standard library (104 modules imported), EucDomain	Mathlib (56 modules imported directly)
Proofs	terms, by pattern matching and equational reasoning; reflection-based solvers	tactics: about 27,000 tactic blocks, 23,000 automation calls
Reused from the sources	the authors' Agda proofs for Clifford+T and Clifford+CS	no proofs; the authors' coset-action data for Clifford+T
Modules	1,477	816
Code lines	375,673	339,594
Verification time		
Decided equalities	a Boolean reduced to `true`	`decide +kernel`
Check time, sixteen rows	2,332 s	12,471 s
Faster for	thirteen rows	three rows
Execution time		
Normal forms	executable functions	existence theorems or `noncomputable` definitions (160 declared)
Benchmarked procedure	matrix of each circuit, by products of stored matrices	simulation of each circuit on every basis vector
Compiled through	GHC (Haskell)	C
Per decision, Clifford, 100 gates	5.0 ms on 2 qubits, 24 s on 6	5.8 ms on 2 qubits, 0.61 s on 6

Table 2. The Agda and Lean formalizations compared. Counts are over the sixteen development folders of Table 1, without the shared libraries, in the current tree; code lines exclude blank lines and comments. Check times are those of Table 1, execution times those of Figure 3(c).

6 Corrections to the Source Papers

The formalizations in Table 1 identify two kinds of issues in the source papers. Most are typos, and some of them appear in statements that the proofs rely on. These are easy to fix. Two others are harder to repair. One is a gap in a proof, and the other is a relation missing from a theorem. We describe both below.

The gap is in the proof of Theorem 4.14 of Hadamard-Π [27], which states that the relations in its Figure 6, over the one- and two-level matrices $(-1)_{[a]}$, $\mathrm{X}_{[a,b]}$, and $\mathrm{H}_{[a,b]}$, present the group $O_n\left(\mathbb{Z}\left[1/\sqrt{2}\right]\right)$. One case of the proof relies on Lemmas A.16 and A.17, which have counterexamples in dimension 6, and the case analysis built on them misses a matrix in dimension 9. The authors have confirmed this gap (personal communication). Their main result is however still correct: Clément's unpublished notes [20] present the same group with different relations, and his case analysis covers the missing case, using

which the formalization of Theorem 4.14 is completed. We also prove that the two sets of relations are equivalent in every dimension and formalize Clément's proof [3].

The relation is missing from Theorem 5.2 of [2]. It states that the relations in its Figures 4 and 7, over the Toffoli and $\mathrm{K} = \mathrm{H} \otimes \mathrm{H}$ gates, present the group $O_8(\mathbb{Z}[1/2])$. This is the first of two steps towards presenting the scaled dyadic orthogonal group L_8 by three-qubit Toffoli–Hadamard circuits. The relation r_K in its Section 5.5, which splits K on two qubits into two copies controlled by the third qubit (**??**), is in neither figure. Nor can it be derived from them. Hence, Theorem 5.2 fails as stated, and we have informed the authors. Our formalization [4] shows that r_K is independent by giving an interpretation of circuits that every other relation preserves but r_K does not, and proves the presentations of $O_8(\mathbb{Z}[1/2])$ and L_8 with r_K restored.

7 Discussion and Future Work

We formalize the results of sixteen papers in QCert, using consistent circuit syntax, a unified definition of group presentations, and shared libraries for rings and their matrix groups. Since all formalizations use these definitions, presentations of the same group from different papers can be compared and proved equivalent. The same libraries, together with the induction that turns an exact synthesis algorithm into a completeness proof, can be reused for new algorithms. Only the case analysis over matrices and generators is specific to each algorithm. As the most error-prone part of the proof, it is where machine checking in QCert matters most.

At the moment, not every completeness result fits into QCert, as its scope is limited in two ways. First, it supports only discrete gate sets, so it does not cover completeness results for circuits with continuous parameters, such as rotations by arbitrary angles [22–24]. Second, control is not a circuit constructor in QCert, so it does not cover results that treat control as a primitive operation on circuits [12, 26, 53].

We plan to extend the infrastructure of QCert to more families of quantum circuits, including the two above. Completeness is still open for many discrete gate sets in higher-dimensional quantum systems, and their case analyses are often too large to check by hand. In particular, we plan to study the completeness of Clifford circuits in all dimensions with the help of QCert. Beyond completeness, the exact synthesis libraries can be reused for T-count optimization of multi-qubit Clifford+T circuits.

References

[1] Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A finite presentation of CNOT-dihedral operators. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL 2017) (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 84–97. doi:10.4204/EPTCS.266.5

[2] Matthew Amy, Neil J. Ross, and Scott Wesley. 2024. A sound and complete equational theory for 3-qubit Toffoli-Hadamard circuits. In *Proceedings of the 21st International Conference on Quantum Physics and Logic (QPL 2024) (Electronic Proceedings in Theoretical Computer Science, Vol. 406)*. 1–43. arXiv:2407.11152 doi:10.4204/EPTCS.406.1

[3] Anonymous. 2026. QCert: The presentations of $O_n(\mathbb{Z}[1/\sqrt{2}])$ by Hadamard-Π and by Clément, formalized in Agda. Supplementary material, modules `Real-Clifford+CH-TwoLevel/Equivalence` (the equivalence of the two sets of relations) and `Real-Clifford+CH-TwoLevel-Clement` (Clément's proof and its decision trees).

[4] Anonymous. 2026. QCert: Three-qubit Toffoli–Hadamard circuits with the relation r_K, formalized in Agda. Supplementary material, modules `CCX+HH-3qubit-opus/Theorems` (the presentations of $O_8(\mathbb{Z}[1/2])$ and L_8) and `CCX+HH-3qubit-opus/Independence` (the interpretation showing that r_K is not derivable).
[5] Anthropic. 2026. Formalizing Fermat's Last Theorem. Research post. https://www.anthropic.com/research/formalizing-fermats-last-theorem 4 September 2026.
[6] Alexander Bastounis, Fabian Circelli, and Anders C. Hansen. 2026. Navier-Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. *arXiv preprint arXiv:2610.08144* (2026).
[7] Xiaoning Bian and Yuan Feng. 2027. Normal forms and complete relations for circuits in Agda. *Proceedings of the ACM on Programming Languages* POPL (2027). To appear.
[8] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A complete and natural rule set for multi-qudit Clifford circuits in all odd prime dimensions. arXiv:2609.40106
[9] Xiaoning Bian and Peter Selinger. 2021. Generators and relations for $U_n(\mathbb{Z}[1/2, i])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 145–164. arXiv:2105.14047 doi:10.4204/EPTCS.343.8
[10] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 2-qubit Clifford+T operators. In *Proceedings of the 19th International Conference on Quantum Physics and Logic (QPL 2022) (Electronic Proceedings in Theoretical Computer Science, Vol. 394)*. 13–28. arXiv:2204.02217 doi:10.4204/EPTCS.394.2
[11] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 3-qubit Clifford+CS operators. In *Proceedings of the 20th International Conference on Quantum Physics and Logic (QPL 2023) (Electronic Proceedings in Theoretical Computer Science, Vol. 384)*. 114–126. arXiv:2306.08530 doi:10.4204/EPTCS.384.7
[12] Colin Blake. 2026. A complete equational presentation of qudit circuits via polycontrolled PROPs. In *Proceedings of the 51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026) (Leibniz International Proceedings in Informatics, Vol. 386)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:18. arXiv:2602.09873 doi:10.4230/LIPIcs.MFCS.2026.6
[13] Colin Blake. 2026. Completeness for prime-dimensional phase-affine circuits. arXiv:2603.06466
[14] Colin Blake. 2026. Simpler presentations for many fragments of quantum circuits. In *Proceedings of the 11th International Conference on Formal Structures for Computation and Deduction (FSCD 2026) (Leibniz International Proceedings in Informatics, Vol. 378)*. 6:1–6:20. arXiv:2602.09874 doi:10.4230/LIPIcs.FSCD.2026.6
[15] Ana Bove, Peter Dybjer, and Ulf Norell. 2009. A brief overview of Agda – a functional language with dependent types. In *Theorem Proving in Higher Order Logics (TPHOLs 2009) (Lecture Notes in Computer Science, Vol. 5674)*. Springer, Berlin, Heidelberg, 73–78.
[16] Joachim Breitner. 2010. Free groups. Archive of Formal Proofs. https://www.isa-afp.org/entries/Free-Groups.html
[17] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a few square roots, quantum computing is as easy as Π. *Proceedings of the ACM on Programming Languages* 8, POPL (2024). arXiv:2310.14056
[18] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An automated deductive verification framework for circuit-building quantum programs. In *Programming Languages and Systems: 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. 148–177.
[19] Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in reversible programming: From symmetric rig groupoids to reversible programming languages. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 6 (2022), 32 pages.
[20] Alexandre Clément. 2019. A complete set of relations for the real Clifford+T group. Unfinished manuscript. https://www.mathstat.dal.ca/~aclement/Case%20distinction.pdf
[21] Alexandre Clément. 2026. A complete equational theory for real-Clifford+CH quantum circuits. arXiv:2602.06644
[22] Alexandre Clément, Noé Delorme, and Simon Perdrix. 2024. Minimal equational theories for quantum circuits. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*.

27:1–27:14. arXiv:2311.07476 doi:10.1145/3661814.3662088

[23] Alexandre Clément, Noé Delorme, Simon Perdrix, and Renaud Vilmart. 2024. Quantum circuit completeness: Extensions and simplifications. In *Proceedings of the 32nd EACSL Annual Conference on Computer Science Logic (CSL 2024) (Leibniz International Proceedings in Informatics, Vol. 288)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 20:1–20:23. arXiv:2303.03117 doi:10.4230/LIPIcs.CSL.2024.20

[24] Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2023. A complete equational theory for quantum circuits. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. 1–13. arXiv:2206.10577 doi:10.1109/LICS56636.2023.10175801

[25] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37

[26] Noé Delorme and Simon Perdrix. 2026. Diagrammatic reasoning with control as a constructor, applications to quantum circuits. In *Foundations of Software Science and Computation Structures (FoSSaCS 2026) (Lecture Notes in Computer Science, Vol. 16503)*. Springer Nature Switzerland, Cham, Switzerland, 240–261. arXiv:2508.21756 doi:10.1007/978-3-032-22730-0_12

[27] Wang Fang, Chris Heunen, and Robin Kaarsgaard. 2026. Hadamard-Π: Equational quantum programming. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 5 (2026), 27 pages. arXiv:2506.06835 doi:10.1145/3776647

[28] Shiguang Feng and Lvzhou Li. 2026. A complete set of transformation rules for reversible circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 8 (2026), 3711–3724. arXiv:2508.17273 doi:10.1109/TCAD.2025.3641533

[29] Shiguang Feng, Zongxing Xiong, and Lvzhou Li. 2026. On the completeness of transformation rules in reversible logic synthesis. arXiv:2607.19059

[30] Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Pasca, Laurence Rideau, Alexey Solovyev, Enrico Tassi, and Laurent Théry. 2013. A machine-checked proof of the odd order theorem. In *Interactive Theorem Proving (ITP 2013) (Lecture Notes in Computer Science, Vol. 7998)*. 163–179.

[31] Seth Eveson Murray Greylyn. 2014. *Generators and relations for the group* $U_4(\mathbb{Z}[1/\sqrt{2}, i])$. MSc thesis. Dalhousie University. arXiv:1408.6204

[32] David Gross. 2006. Hudson's theorem for finite-dimensional quantum systems. *J. Math. Phys.* 47, 12 (2006), 122107. doi:10.1063/1.2393152

[33] Shamgar Gurevich and Ronny Hadani. 2007. The geometric Weil representation. *Selecta Mathematica* 13 (2007), 465–481. doi:10.1007/s00029-007-0047-3

[34] Shamgar Gurevich and Ronny Hadani. 2010. Bounds on certain higher-dimensional exponential sums via the self-reducibility of the Weil representation. arXiv:math/0612765 [math.RT] https://arxiv.org/abs/math/0612765

[35] Thomas Hales. 2026. What mathematicians should know about the Lean theorem prover: Questions of reliability and AI. Guest post on Terence Tao's blog *What's new*. https://terrytao.wordpress.com/2026/10/09/what-mathematicians-should-know-about-the-lean-theorem-proverquestions-of-reliability-and-ai/ 9 October 2026.

[36] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (2021), 29 pages. doi:10.1145/3434318

[37] Aabid Seeyal Abdul Kharim, Turga Venkata Hanumantha Prathamesh, Shweta Rajiv, and Rishi Vyas. 2023. Formalizing free groups in Isabelle/HOL: The Nielsen-Schreier theorem and the conjugacy problem. In *Intelligent Computer Mathematics (CICM 2023) (Lecture Notes in Computer Science, Vol. 14101)*.

[38] Yves Lafont. 2003. Towards an algebraic theory of Boolean circuits. *Journal of Pure and Applied Algebra* 184, 2–3 (2003), 257–310. doi:10.1016/S0022-4049(03)00069-0

[39] Adrian Lehmann, Ben Caldwell, Bhakti Shah, William Spencer, and Robert Rand. 2026. VyZX: Formal verification of a graphical quantum language. *ACM Transactions on Programming Languages and Systems* (2026). arXiv:2311.11571

[40] Liyi Li, Finn Voichick, Kesha Hietala, Yuxiang Peng, Xiaodi Wu, and Michael Hicks. 2022. Verified Compilation of Quantum Oracles. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 146:1–146:27. doi:10.1145/3563309

[41] Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A complete and natural rule set for multi-qutrit Clifford circuits. In *Proceedings of the 22nd International Conference on Quantum Physics and Logic (QPL 2025) (Electronic Proceedings in Theoretical Computer Science, Vol. 426)*. 23–78. arXiv:2508.14670 doi:10.4204/EPTCS.426.2

[42] Sarah Meng Li, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for the group $O_n(\mathbb{Z}[1/2])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 210–264. arXiv:2106.01175 doi:10.4204/EPTCS.343.11

[43] Saunders Mac Lane. 1965. Categorical algebra. *Bull. Amer. Math. Soc.* 71, 1 (1965), 40–106. doi:10.1090/S0002-9904-1965-11234-4

[44] Wilhelm Magnus, Abraham Karrass, and Donald Solitar. 2004. *Combinatorial group theory: Presentations of groups in terms of generators and relations* (2nd revised ed.). Dover Publications.

[45] Justin Makary, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for real stabilizer operators. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 14–36. arXiv:2109.05655 doi:10.4204/EPTCS.343.2

[46] Dmitri Maslov, Gerhard W. Dueck, D. Michael Miller, and Camille Negrevergne. 2008. Quantum circuit simplification and level compaction. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 3 (2008), 436–444. doi:10.1109/TCAD.2007.911334

[47] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. 2018. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information* 4 (2018), 23. doi:10.1038/s41534-018-0072-4

[48] Markus Neuhauser. 2002. An Explicit Construction of the Metaplectic Representation over a Finite Field. *Journal of Lie Theory* 12, 1 (2002), 15–30. doi:10.5802/jolt.250

[49] Ulf Norell. 2007. *Towards a practical programming language based on dependent type theory*. Ph. D. Dissertation. Chalmers University of Technology.

[50] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A core language for quantum circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*. 846–858.

[51] Anurudh Peduri, Ina Schaefer, and Michael Walter. 2025. QbC: Quantum Correctness by Construction. *Proceedings of the ACM on Programming Languages* 9, OOPSLA1 (2025). doi:10.1145/3720433

[52] Issei Sakashita. 2013. A complete transformation rule set and a minimal equation set for CNOT-based 3-qubit quantum circuits (Draft). arXiv:1304.5916

[53] William Schober and Scott Wesley. 2026. A complete equational theory for quantum circuits with generalized control. https://cqi.inf.usi.ch/publications/Hierarchical_QPL2026.pdf Accepted to the proceedings track of the 23rd International Conference on Quantum Physics and Logic (QPL 2026).

[54] Peter Selinger. 2015. Generators and relations for n-qubit Clifford operators. *Logical Methods in Computer Science* 11, 2:10 (2015). arXiv:1310.6813 doi:10.2168/LMCS-11(2:10)2015

[55] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. t|ket⟩: A retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (2020), 014003. doi:10.1088/2058-9565/ab8e92

[56] The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. 367–381. doi:10.1145/3372885.3373824

[57] The mathlib Community. 2026. Mathlib documentation: PresentedGroup, CoxeterGroup, and the Nielsen-Schreier theorem. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/FreeGroup/NielsenSchreier.html

[58] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A. Acar, and Zhihao Jia. 2022. Quartz: Superoptimization of quantum circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and*

Implementation (PLDI 2022). Association for Computing Machinery, New York, NY, USA, 625–640. doi:10.1145/3519939.3523433

[59] Shigeru Yamashita and Igor L. Markov. 2010. Fast equivalence-checking for quantum circuits. *Quantum Information and Computation* 10, 9–10 (2010), 721–734. doi:10.26421/QIC10.9-10-1

[60] Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational verification of quantum programs. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 29 (2023), 33 pages.